CLIP: Zero-Shot vs Few-Shot Classification on MS COCO 2014

CO5085 — Deep Learning and Applications in Computer Vision

Compare CLIP-based classification strategies on **MS COCO 2014** with **12 supercategory labels** across **three dataset splits** (train / val / test).

Strategy	Training Data	Method
Zero-shot (image-only)	0 labeled samples	CLIP image features vs text prompts
Zero-shot (caption-only)	0 labeled samples	CLIP caption features vs text prompts
Zero-shot (fusion)	0 labeled samples	Avg(image + caption) — true multimodal
Few-shot (N=1,5,10,20)	N images/class from train_pool	Logistic regression on CLIP image features

Dataset: MS COCO 2014 — train2014 (~~83K images) + val2014 (~~41K images), each with 5 human captions, 12 COCO supercategory labels.

Shared Drive: All users share the same dataset folder to avoid redundant downloads. See configuration cell for setup instructions.

```
!pip install -q open-clip-torch pycocotools pandas seaborn tqdm
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 47.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.8/44.8 kB 4.7 MB/s eta 0:00:00
```

```
# ============================================================
# CONFIGURATION
# ============================================================

# ---- Shared Google Drive Setup -------------------------------
# STEP 1: Open the shared folder link below and click
#         'Add shortcut to Drive' -> place it in 'My Drive'.
#   https://drive.google.com/drive/folders/1UVs9LM9N7H_R_cKbr6pNjAGKEgqHzgUp
#
# STEP 2: Set SHARED_FOLDER to the shortcut name you chose.
#         Default expected name: 'deep-learning-asm01'
#
# Everyone shares the same downloaded dataset -- no re-downloading needed.

SHARED_FOLDER = 'deep-learning-asm01'
GDRIVE_BASE   = f'/content/drive/MyDrive/{SHARED_FOLDER}'
COCO_DIR      = f'{GDRIVE_BASE}/data/coco2014'   # shared dataset location
RESULTS_DIR   = f'{GDRIVE_BASE}/results/zeroshot_vs_fewshot'

# ---- COCO 2014 Split Strategy -------------------------------
# Option A (default, DOWNLOAD_TRAIN2014=False):
#   Download only val2014 (~6 GB). Split it into:
#     train_pool (40%) | val (30%) | test (30%)
#
# Option B (DOWNLOAD_TRAIN2014=True):
#   Download train2014 (~13 GB) as the few-shot pool.
#   Split val2014 into val (60%) | test (40%).
#   Gives a much larger training pool at the cost of ~13 GB extra storage.

DOWNLOAD_TRAIN2014 = True

# Fractions for val2014 split (Option A) or val2014->val/test (Option B)
SPLIT_TRAIN = 0.40
SPLIT_VAL   = 0.30
SPLIT_TEST  = 0.30

# ---- CLIP & Few-Shot ----------------------------------------
CLIP_MODEL      = 'ViT-B-32'
CLIP_PRETRAINED = 'openai'
BATCH_SIZE      = 256
FEW_SHOT_COUNTS = [1, 5, 10, 20]
FEW_SHOT_SEEDS  = [0, 1, 2]   # average over 3 seeds

# ---- 12 COCO Supercategories --------------------------------
COCO_SUPERCLASSES = [
    'person', 'vehicle', 'outdoor', 'animal',
    'accessory', 'sports', 'kitchen', 'food',
    'furniture', 'electronic', 'appliance', 'indoor'
]
NUM_CLASSES = len(COCO_SUPERCLASSES)
```

```
SUPER2IDX   = {s: i for i, s in enumerate(COCO_SUPERCLASSES)}

import os

print('Configuration:')
print(f'  Shared Drive  : {GDRIVE_BASE}')
print(f'  COCO dir      : {COCO_DIR}')
print(f'  Train2014     : {"download (~13 GB)" if DOWNLOAD_TRAIN2014 else "skip -- use val2014 split"}')
print(f'  Split ratios  : train={SPLIT_TRAIN:.0%}, val={SPLIT_VAL:.0%}, test={SPLIT_TEST:.0%}')
print(f'  CLIP model    : {CLIP_MODEL} ({CLIP_PRETRAINED})')
print(f'  Few-shot N    : {FEW_SHOT_COUNTS}')
```

```
Configuration:
  Shared Drive  : /content/drive/MyDrive/deep-learning-asm01
  COCO dir      : /content/drive/MyDrive/deep-learning-asm01/data/coco2014
  Train2014     : download (~13 GB)
  Split ratios  : train=40%, val=30%, test=30%
  CLIP model    : ViT-B-32 (openai)
  Few-shot N    : [1, 5, 10, 20]
```

```
# Mount Google Drive and verify shared folder access
import os
from google.colab import drive

if not os.path.isdir('/content/drive/MyDrive'):
    drive.mount('/content/drive')

if os.path.isdir(GDRIVE_BASE):
    os.makedirs(COCO_DIR, exist_ok=True)
    os.makedirs(RESULTS_DIR, exist_ok=True)
    print(f'Shared Drive accessible : {GDRIVE_BASE}')
    print(f'COCO dataset directory  : {COCO_DIR}')
    USE_SHARED_DRIVE = True
else:
    print(f'WARNING: Shared folder not found at {GDRIVE_BASE}')
    print('To fix this:')
    print('  1. Open: https://drive.google.com/drive/folders/1UVs9LM9N7H_R_cKbr6pNjAGKEgqHzgUp')
    print('  2. Right-click the folder -> Add shortcut to Drive -> My Drive')
    print(f'  3. Rename the shortcut to "{SHARED_FOLDER}"')
    print('Falling back to local /content storage (not shared, re-downloads each session).')
    COCO_DIR = '/content/data/coco2014'
    os.makedirs(COCO_DIR, exist_ok=True)
    USE_SHARED_DRIVE = False

print(f'\nUsing COCO dir: {COCO_DIR}')
```

```
Shared Drive accessible : /content/drive/MyDrive/deep-learning-asm01
COCO dataset directory  : /content/drive/MyDrive/deep-learning-asm01/data/coco2014

Using COCO dir: /content/drive/MyDrive/deep-learning-asm01/data/coco2014
```

```
# Download COCO 2014 dataset to the shared Drive (skipped if already present)
import os, zipfile

val_img_dir    = os.path.join(COCO_DIR, 'val2014')
ann_dir        = os.path.join(COCO_DIR, 'annotations')
instances_val  = os.path.join(ann_dir, 'instances_val2014.json')
captions_val   = os.path.join(ann_dir, 'captions_val2014.json')

if DOWNLOAD_TRAIN2014:
    train_img_dir   = os.path.join(COCO_DIR, 'train2014')
    instances_train = os.path.join(ann_dir, 'instances_train2014.json')
    captions_train  = os.path.join(ann_dir, 'captions_train2014.json')

def wget_extract(url, tmp, dest, label):
    print(f'Downloading {label} ...')
    os.system(f'wget -q "{url}" -O "{tmp}"')
    print(f'Extracting {label} ...')
    with zipfile.ZipFile(tmp) as z:
        z.extractall(dest)
    os.remove(tmp)
    print(f'Done: {label}')

# val2014 images (~6 GB)
if not os.path.isdir(val_img_dir) or len(os.listdir(val_img_dir)) < 10:
    wget_extract(
        'http://images.cocodataset.org/zips/val2014.zip',
```

```
        '/tmp/val2014.zip', COCO_DIR, 'val2014 images (~6 GB)'
    )
else:
    print(f'val2014: {len(os.listdir(val_img_dir)):,} images found in {val_img_dir}')

# Annotations (contains both train2014 and val2014 annotations)
if not os.path.isfile(instances_val) or not os.path.isfile(captions_val):
    wget_extract(
        'http://images.cocodataset.org/annotations/annotations_trainval2014.zip',
        '/tmp/anns2014.zip', COCO_DIR, 'annotations_trainval2014'
    )
else:
    print('Annotations: already present')

# Optional: train2014 images (~13 GB)
if DOWNLOAD_TRAIN2014:
    if not os.path.isdir(train_img_dir) or len(os.listdir(train_img_dir)) < 10:
        wget_extract(
            'http://images.cocodataset.org/zips/train2014.zip',
            '/tmp/train2014.zip', COCO_DIR, 'train2014 images (~13 GB)'
        )
    else:
        print(f'train2014: {len(os.listdir(train_img_dir)):,} images found')

print('\nDataset ready.')
```

```
val2014: 40,504 images found in /content/drive/MyDrive/deep-learning-asm01/data/coco2014/val2014
Annotations: already present
train2014: 82,783 images found

Dataset ready.
```

```
# Parse COCO 2014 annotations with pycocotools
from pycocotools.coco import COCO

print('Loading val2014 annotations...')
coco_inst_val = COCO(instances_val)
coco_caps_val = COCO(captions_val)

if DOWNLOAD_TRAIN2014:
    print('Loading train2014 annotations...')
    coco_inst_train = COCO(instances_train)
    coco_caps_train = COCO(captions_train)

# Build category_id -> supercategory mapping (identical for both splits)
cat_to_super = {
    cat['id']: cat['supercategory']
    for cat in coco_inst_val.loadCats(coco_inst_val.getCatIds())
}

supers_in_coco = sorted(set(cat_to_super.values()))
print(f'\nFine-grained categories : {len(coco_inst_val.getCatIds())}')
print(f'Supercategories in COCO : {supers_in_coco}')
missing = [s for s in COCO_SUPERCLASSES if s not in supers_in_coco]
if missing:
    print(f'WARNING -- supercategories missing from COCO: {missing}')
else:
    print('All 12 target supercategories confirmed in COCO 2014.')
```

```
Loading val2014 annotations...
loading annotations into memory...
Done (t=4.33s)
creating index...
index created!
loading annotations into memory...
Done (t=0.34s)
creating index...
index created!
Loading train2014 annotations...
loading annotations into memory...
Done (t=12.03s)
creating index...
index created!
loading annotations into memory...
Done (t=2.11s)
creating index...
index created!

Fine-grained categories : 80
```

```
Supercategories in COCO : ['accessory', 'animal', 'appliance', 'electronic', 'food', 'furniture', 'indoor', 'kitchen', 'outdoor'
All 12 target supercategories confirmed in COCO 2014.
```

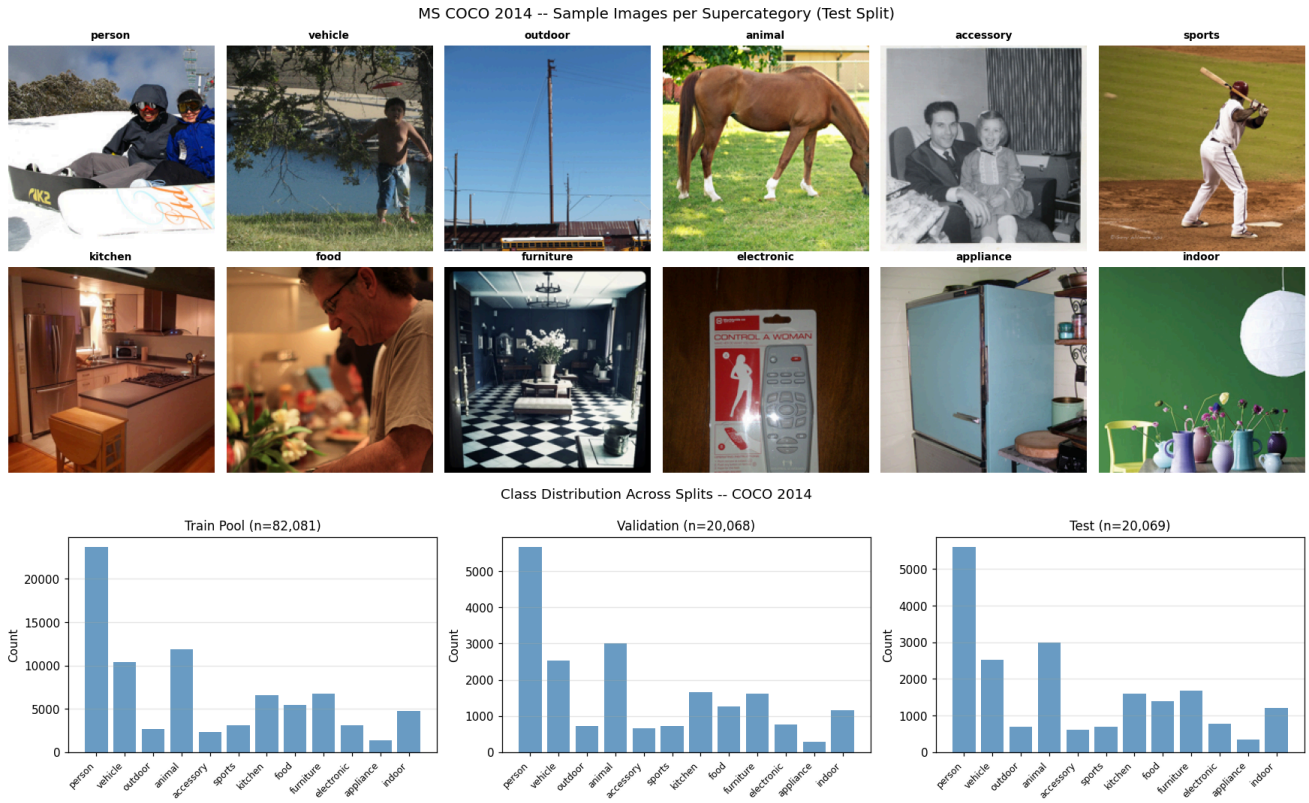
```
# Build labeled records and create train / val / test splits
from collections import Counter
from tqdm.auto import tqdm
import numpy as np, os

def get_dominant_super(img_id, coco_inst_obj):
    anns   = coco_inst_obj.loadAnns(coco_inst_obj.getAnnIds(imgIds=img_id))
    supers = [cat_to_super.get(a['category_id'], '') for a in anns]
    supers = [s for s in supers if s in SUPER2IDX]
    return Counter(supers).most_common(1)[0][0] if supers else None

def get_caps(img_id, coco_caps_obj):
    return [a['caption'] for a in coco_caps_obj.loadAnns(
        coco_caps_obj.getAnnIds(imgIds=img_id))]

def build_records(img_ids, img_dir, coco_inst_obj, coco_caps_obj, desc):
    records = []
    for img_id in tqdm(sorted(img_ids), desc=desc):
        sup  = get_dominant_super(img_id, coco_inst_obj)
        caps = get_caps(img_id, coco_caps_obj)
        if sup is None or not caps:
            continue
        info     = coco_inst_obj.loadImgs(img_id)[0]
        img_path = os.path.join(img_dir, info['file_name'])
        if not os.path.isfile(img_path):
            continue
        records.append({
            'img_id'       : img_id,
            'img_path'     : img_path,
            'label'        : SUPER2IDX[sup],
            'supercategory': sup,
            'captions'     : caps[:5]
        })
    return records

# Always build val2014 records
print('Building records from val2014...')
val2014_all = build_records(
    coco_inst_val.getImgIds(), val_img_dir,
    coco_inst_val, coco_caps_val, 'val2014'
)
print(f'val2014 labeled records: {len(val2014_all):,}')

if DOWNLOAD_TRAIN2014:
    # Option B: train2014 as pool, val2014 -> val + test
    print('Building records from train2014...')
    train2014_all = build_records(
        coco_inst_train.getImgIds(), train_img_dir,
        coco_inst_train, coco_caps_train, 'train2014'
    )
    print(f'train2014 labeled records: {len(train2014_all):,}')
    n_val     = int(len(val2014_all) * SPLIT_VAL / (SPLIT_VAL + SPLIT_TEST))
    train_data = train2014_all
    val_data   = val2014_all[:n_val]
    test_data  = val2014_all[n_val:]
    split_mode = 'train2014 pool + val2014 val/test'
else:
    # Option A: split val2014 three ways
    N          = len(val2014_all)
    n_train    = int(N * SPLIT_TRAIN)
    n_val      = int(N * SPLIT_VAL)
    train_data = val2014_all[:n_train]
    val_data   = val2014_all[n_train : n_train + n_val]
    test_data  = val2014_all[n_train + n_val:]
    split_mode = f'val2014 split {SPLIT_TRAIN:.0%}/{SPLIT_VAL:.0%}/{SPLIT_TEST:.0%}'

print(f'\nSplit mode : {split_mode}')
print(f'train_pool : {len(train_data):,} images')
print(f'val        : {len(val_data):,} images')
print(f'test       : {len(test_data):,} images')

# Per-split class distribution
print('\nPer-class image counts:')
```

```
print(f'{"Supercategory":14s}  {"train_pool":>10s}  {"val":>8s}  {"test":>8s}')
print('-' * 46)
for sup in COCO_SUPERCLASSES:
    tr = Counter(d['supercategory'] for d in train_data).get(sup, 0)
    va = Counter(d['supercategory'] for d in val_data).get(sup, 0)
    te = Counter(d['supercategory'] for d in test_data).get(sup, 0)
    print(f'{sup:14s}  {tr:>10,}  {va:>8,}  {te:>8,}')
```

```
Building records from val2014...
val2014: 100%                                          40504/40504 [00:12<00:00, 2969.38it/s]
val2014 labeled records: 40,137
Building records from train2014...
train2014: 100%                                        82783/82783 [00:24<00:00, 3632.79it/s]
train2014 labeled records: 82,081

Split mode : train2014 pool + val2014 val/test
train_pool : 82,081 images
val        : 20,068 images
test       : 20,069 images

Per-class image counts:
Supercategory   train_pool       val      test
----------------------------------------------
person              23,647     5,664     5,594
vehicle             10,414     2,540     2,522
outdoor              2,642       729       694
animal              11,857     3,005     2,982
accessory            2,353       649       611
sports               3,074       725       691
kitchen              6,598     1,648     1,589
food                 5,476     1,265     1,392
furniture            6,745     1,618     1,680
electronic           3,114       765       767
appliance            1,384       292       349
indoor               4,777     1,168     1,198
```

```
# EDA: sample images + class distribution across splits
import matplotlib.pyplot as plt, matplotlib
from PIL import Image
from collections import Counter
import numpy as np
matplotlib.rcParams['figure.dpi'] = 110

# Sample one image per supercategory (from test split)
fig, axes = plt.subplots(2, 6, figsize=(16, 6))
axes = axes.flatten()
shown = {}
for d in test_data:
    sup = d['supercategory']
    if sup not in shown:
        try:
            img = Image.open(d['img_path']).convert('RGB')
            shown[sup] = (img, d['captions'][0])
        except Exception:
            pass
    if len(shown) == NUM_CLASSES:
        break
for i, sup in enumerate(COCO_SUPERCLASSES):
    if sup in shown:
        img, cap = shown[sup]
        w, h = img.size
        side = min(w, h)
        axes[i].imshow(img.crop((0, 0, side, side)).resize((200, 200)))
        cap_short = cap[:45] + '...' if len(cap) > 45 else cap
        axes[i].set_title(sup, fontsize=9, fontweight='bold')
        axes[i].set_xlabel(cap_short, fontsize=6)
    axes[i].axis('off')
plt.suptitle('MS COCO 2014 -- Sample Images per Supercategory (Test Split)', fontsize=13)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'samples.png'), bbox_inches='tight')
plt.show()

# Class distribution across 3 splits
fig, axes2 = plt.subplots(1, 3, figsize=(16, 4))
for ax, split, name in [
    (axes2[0], train_data, 'Train Pool'),
```

```
    (axes2[1], val_data,   'Validation'),
    (axes2[2], test_data,  'Test')
]:
    counts = Counter(d['supercategory'] for d in split)
    ax.bar(range(NUM_CLASSES), [counts.get(s, 0) for s in COCO_SUPERCLASSES],
           color='steelblue', alpha=0.8)
    ax.set_xticks(range(NUM_CLASSES))
    ax.set_xticklabels(COCO_SUPERCLASSES, rotation=45, ha='right', fontsize=8)
    ax.set_title(f'{name} (n={len(split):,})', fontsize=11)
    ax.set_ylabel('Count')
    ax.grid(True, axis='y', alpha=0.3)
plt.suptitle('Class Distribution Across Splits -- COCO 2014', fontsize=12)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'class_distribution.png'), bbox_inches='tight')
plt.show()
print('Saved: samples.png, class_distribution.png')
```




```
Saved: samples.png, class_distribution.png
```

```
# Load CLIP ViT-B/32 (OpenAI pretrained)
import torch, open_clip

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Device: {device}')
if torch.cuda.is_available():
    print(f'GPU   : {torch.cuda.get_device_name(0)}')

model, _, preprocess = open_clip.create_model_and_transforms(
    CLIP_MODEL, pretrained=CLIP_PRETRAINED
)
```

```
tokenizer = open_clip.get_tokenizer(CLIP_MODEL)
model = model.to(device).eval()

total_params = sum(p.numel() for p in model.parameters()) / 1e6
print(f'CLIP {CLIP_MODEL} ({CLIP_PRETRAINED}): {total_params:.1f}M parameters')
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Device: cuda
GPU   : NVIDIA A100-SXM4-80GB
/usr/local/lib/python3.12/dist-packages/open_clip/factory.py:450: UserWarning: QuickGELU mismatch between final model config (qu
  warnings.warn(
CLIP ViT-B-32 (openai): 151.3M parameters
```

```
# Extract CLIP image and caption features for all three splits
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import numpy as np, torch
from tqdm import tqdm

class COCODataset(Dataset):
    def __init__(self, records, transform):
        self.records   = records
        self.transform = transform
    def __len__(self):
        return len(self.records)
    def __getitem__(self, idx):
        r   = self.records[idx]
        img = Image.open(r['img_path']).convert('RGB')
        return self.transform(img), r['label']

def extract_img(records, label):
    ds     = COCODataset(records, preprocess)
    loader = DataLoader(ds, batch_size=BATCH_SIZE, num_workers=12, pin_memory=False)
    feats, labels = [], []
    with torch.no_grad():
        for imgs, lbls in tqdm(loader, desc=f'  {label} img', leave=False):
            f = model.encode_image(imgs.to(device))
            f = f / f.norm(dim=-1, keepdim=True)
            feats.append(f.cpu().float())
            labels.extend(lbls.tolist())
    return torch.cat(feats, dim=0).numpy(), np.array(labels)

def extract_cap(records, label):
    cap_feats = []
    with torch.no_grad():
        for r in tqdm(records, desc=f'  {label} cap', leave=False):
            tokens = tokenizer(r['captions'][:5]).to(device)
            f      = model.encode_text(tokens)
            f      = f / f.norm(dim=-1, keepdim=True)
            cap_feats.append(f.mean(dim=0).cpu().float().numpy())
    cap_feats = np.stack(cap_feats)
    norms     = np.linalg.norm(cap_feats, axis=1, keepdims=True)
    return cap_feats / (norms + 1e-8)

print('Extracting CLIP features for all splits...')
print('Train pool:')
train_img, train_labels = extract_img(train_data, 'train')
train_cap               = extract_cap(train_data, 'train')

print('Validation:')
val_img, val_labels = extract_img(val_data, 'val')
val_cap             = extract_cap(val_data, 'val')

print('Test:')
test_img, test_labels = extract_img(test_data, 'test')
test_cap              = extract_cap(test_data, 'test')

print('\nFeature shapes:')
print(f'  train_pool : img={train_img.shape}, cap={train_cap.shape}')
print(f'  val        : img={val_img.shape},   cap={val_cap.shape}')
print(f'  test       : img={test_img.shape},  cap={test_cap.shape}')
```

```
Extracting CLIP features for all splits...
Train pool:
Validation:
Test:
```

```
Feature shapes:
  train_pool : img=(82081, 512), cap=(82081, 512)
  val        : img=(20068, 512),   cap=(20068, 512)
  test       : img=(20069, 512),  cap=(20069, 512)
```

```
# Zero-Shot Classification on val and test (3 variants each)
from sklearn.metrics import accuracy_score, f1_score
import numpy as np, torch

PROMPT_TEMPLATES = [
    'a photo of a {cls}.',
    'a picture of a {cls}.',
    'an image containing a {cls}.',
    'a photo showing {cls}.',
    '{cls} in a photograph.'
]

def build_text_embs(classnames, templates):
    embs = []
    with torch.no_grad():
        for cls in classnames:
            tokens = tokenizer([t.format(cls=cls) for t in templates]).to(device)
            f      = model.encode_text(tokens)
            f      = f / f.norm(dim=-1, keepdim=True)
            embs.append(f.mean(dim=0))
    text_embs = torch.stack(embs)
    text_embs = text_embs / text_embs.norm(dim=-1, keepdim=True)
    return text_embs.cpu().float().numpy()

def zs_predict(features, text_embeddings):
    return (features @ text_embeddings.T).argmax(axis=1)

def fuse(img_f, cap_f):
    f = (img_f + cap_f) / 2
    return f / (np.linalg.norm(f, axis=1, keepdims=True) + 1e-8)

def evaluate(preds, labels):
    return (accuracy_score(labels, preds),
            f1_score(labels, preds, average='macro', zero_division=0))

print(f'Building text embeddings ({len(PROMPT_TEMPLATES)} prompts x {NUM_CLASSES} classes)...')
text_embs = build_text_embs(COCO_SUPERCLASSES, PROMPT_TEMPLATES)

zs_results = {}
for split_name, img_f, cap_f, labels in [
    ('val',  val_img,  val_cap,  val_labels),
    ('test', test_img, test_cap, test_labels)
]:
    preds_img = zs_predict(img_f, text_embs)
    preds_cap = zs_predict(cap_f, text_embs)
    preds_fus = zs_predict(fuse(img_f, cap_f), text_embs)
    a_img, f_img = evaluate(preds_img, labels)
    a_cap, f_cap = evaluate(preds_cap, labels)
    a_fus, f_fus = evaluate(preds_fus, labels)
    zs_results[split_name] = {
        'image_only'  : {'acc': a_img, 'f1': f_img, 'preds': preds_img, 'labels': labels},
        'caption_only': {'acc': a_cap, 'f1': f_cap, 'preds': preds_cap, 'labels': labels},
        'fusion'      : {'acc': a_fus, 'f1': f_fus, 'preds': preds_fus, 'labels': labels}
    }
    print(f'\n--- Zero-Shot [{split_name.upper()}, n={len(labels):,}] ---')
    print(f'  Image-only   : Acc={a_img:.4f}  F1={f_img:.4f}')
    print(f'  Caption-only : Acc={a_cap:.4f}  F1={f_cap:.4f}')
    print(f'  Fusion       : Acc={a_fus:.4f}  F1={f_fus:.4f}')
```

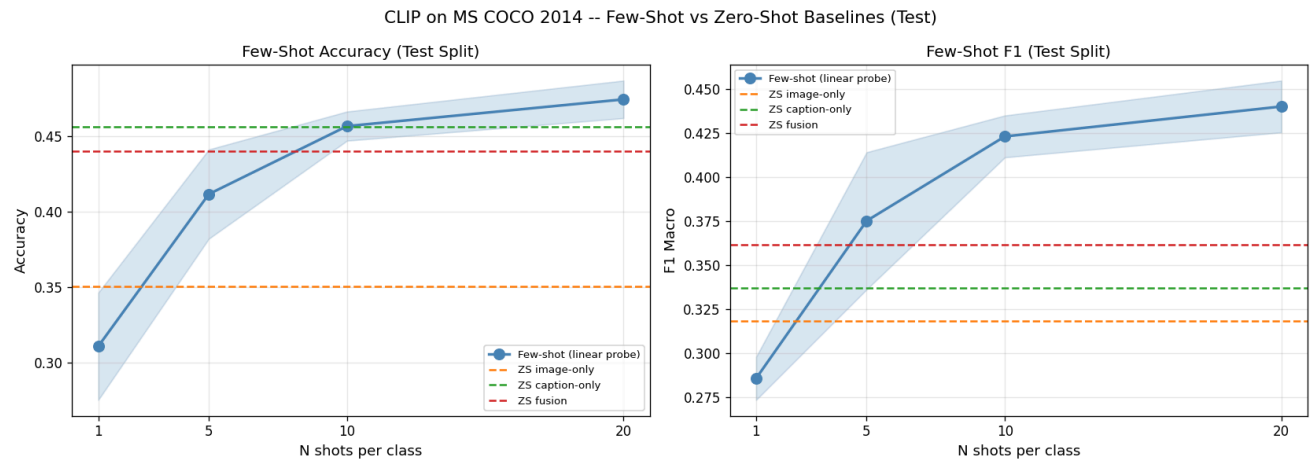
```
Building text embeddings (5 prompts x 12 classes)...

--- Zero-Shot [VAL, n=20,068] ---
  Image-only   : Acc=0.3472  F1=0.3148
  Caption-only : Acc=0.4565  F1=0.3367
  Fusion       : Acc=0.4427  F1=0.3591

--- Zero-Shot [TEST, n=20,069] ---
  Image-only   : Acc=0.3503  F1=0.3181
  Caption-only : Acc=0.4558  F1=0.3368
  Fusion       : Acc=0.4397  F1=0.3616
```

```python
# Few-Shot Classification: train_pool -> evaluate on val and test
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import normalize
import numpy as np

def few_shot_eval(train_f, train_lbl, eval_f, eval_lbl, n_shot, seed):
    rng  = np.random.default_rng(seed)
    idx  = []
    for cls in range(NUM_CLASSES):
        cls_idx = np.where(train_lbl == cls)[0]
        chosen  = rng.choice(cls_idx, min(n_shot, len(cls_idx)), replace=False)
        idx.extend(chosen.tolist())
    clf = LogisticRegression(max_iter=1000, C=1.0, random_state=int(seed))
    clf.fit(normalize(train_f[idx]), train_lbl[idx])
    preds = clf.predict(normalize(eval_f))
    return (accuracy_score(eval_lbl, preds),
            f1_score(eval_lbl, preds, average='macro', zero_division=0),
            preds)

print('Few-shot experiments (train_pool -> val and test)...')
print(f'{"N":>4}  {"Val Acc":>12}  {"Val F1":>12}  {"Test Acc":>12}  {"Test F1":>12}')
print('-' * 58)

fs_results = {'val': {}, 'test': {}}
best_n = max(FEW_SHOT_COUNTS)
best_val_preds, best_test_preds = None, None

for n in FEW_SHOT_COUNTS:
    va_accs, va_f1s, te_accs, te_f1s = [], [], [], []
    for seed in FEW_SHOT_SEEDS:
        va, vf, vp = few_shot_eval(train_img, train_labels, val_img,  val_labels,  n, seed)
        ta, tf, tp = few_shot_eval(train_img, train_labels, test_img, test_labels, n, seed)
        va_accs.append(va); va_f1s.append(vf)
        te_accs.append(ta); te_f1s.append(tf)
        if n == best_n and seed == FEW_SHOT_SEEDS[0]:
            best_val_preds, best_test_preds = vp, tp
    for split, accs, f1s in [('val', va_accs, va_f1s), ('test', te_accs, te_f1s)]:
        fs_results[split][n] = {
            'acc': float(np.mean(accs)), 'acc_std': float(np.std(accs)),
            'f1' : float(np.mean(f1s)),  'f1_std' : float(np.std(f1s))
        }
    print(f'{n:>4}  '
          f'{np.mean(va_accs):.4f}+/-{np.std(va_accs):.4f}  '
          f'{np.mean(va_f1s):.4f}+/-{np.std(va_f1s):.4f}  '
          f'{np.mean(te_accs):.4f}+/-{np.std(te_accs):.4f}  '
          f'{np.mean(te_f1s):.4f}+/-{np.std(te_f1s):.4f}')
```

```
Few-shot experiments (train_pool -> val and test)...
   N       Val Acc        Val F1      Test Acc       Test F1
----------------------------------------------------------
   1  0.3041+/-0.0343  0.2758+/-0.0141  0.3107+/-0.0355  0.2857+/-0.0122
   5  0.4073+/-0.0291  0.3717+/-0.0357  0.4115+/-0.0295  0.3752+/-0.0389
  10  0.4527+/-0.0053  0.4200+/-0.0080  0.4565+/-0.0096  0.4231+/-0.0119
  20  0.4698+/-0.0124  0.4354+/-0.0144  0.4742+/-0.0124  0.4401+/-0.0147
```

```python
# Few-Shot Curve: accuracy and F1 vs N shots on TEST split
import matplotlib.pyplot as plt, matplotlib
matplotlib.rcParams['figure.dpi'] = 120

ns        = FEW_SHOT_COUNTS
acc_means = [fs_results['test'][n]['acc']     for n in ns]
acc_stds  = [fs_results['test'][n]['acc_std'] for n in ns]
f1_means  = [fs_results['test'][n]['f1']      for n in ns]
f1_stds   = [fs_results['test'][n]['f1_std']  for n in ns]

zs_test = zs_results['test']
baselines = [
    (zs_test['image_only']['acc'],   zs_test['image_only']['f1'],   'ZS image-only',   '#ff7f0e'),
    (zs_test['caption_only']['acc'], zs_test['caption_only']['f1'], 'ZS caption-only', '#2ca02c'),
    (zs_test['fusion']['acc'],       zs_test['fusion']['f1'],       'ZS fusion',       '#d62728'),
]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
for ax, means, stds, idx, ylabel, title in [
    (ax1, acc_means, acc_stds, 0, 'Accuracy', 'Few-Shot Accuracy (Test Split)'),
    (ax2, f1_means,  f1_stds,  1, 'F1 Macro', 'Few-Shot F1 (Test Split)')
]:
```

```
    lo = [m - s for m, s in zip(means, stds)]
    hi = [m + s for m, s in zip(means, stds)]
    ax.fill_between(ns, lo, hi, alpha=0.2, color='steelblue')
    ax.plot(ns, means, 'o-', color='steelblue', lw=2, ms=8, label='Few-shot (linear probe)')
    for ba, bf, lbl, clr in baselines:
        ax.axhline(ba if idx == 0 else bf, linestyle='--', color=clr, lw=1.5, label=lbl)
    ax.set_xlabel('N shots per class', fontsize=11)
    ax.set_ylabel(ylabel, fontsize=11)
    ax.set_title(title, fontsize=12)
    ax.set_xticks(ns)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

plt.suptitle('CLIP on MS COCO 2014 -- Few-Shot vs Zero-Shot Baselines (Test)', fontsize=13)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'few_shot_curve.png'), bbox_inches='tight')
plt.show()
print('Saved: few_shot_curve.png')
```

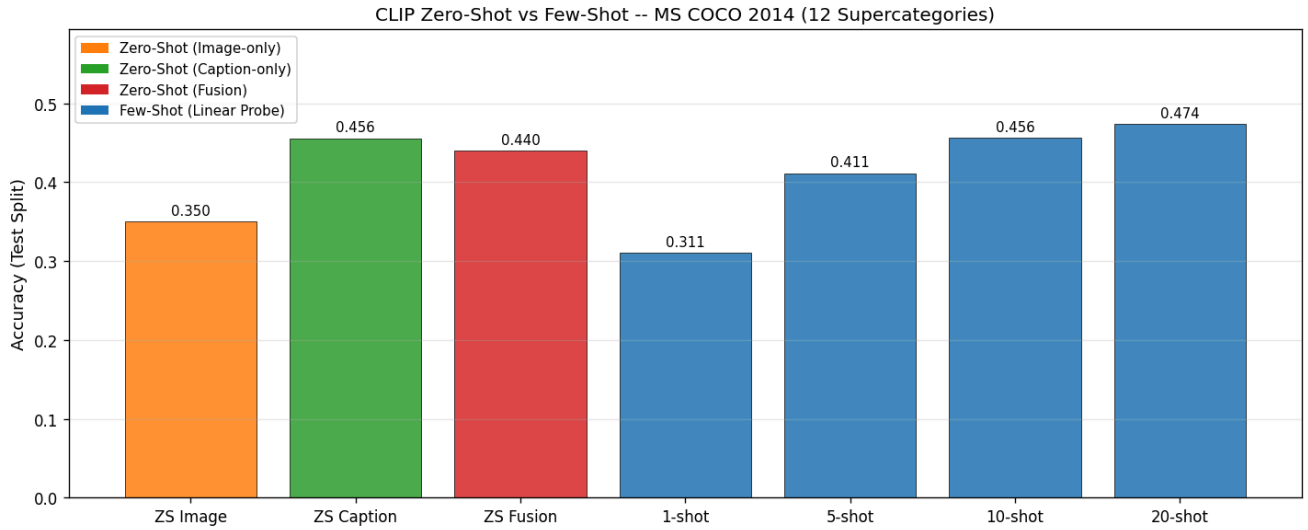



```
Saved: few_shot_curve.png
```

```
# Full Comparison Table (Val + Test) and Bar Chart
import pandas as pd, matplotlib.pyplot as plt
from matplotlib.patches import Patch

print('=' * 82)
print('FULL RESULTS -- MS COCO 2014 (12 supercategories)')
print('=' * 82)

rows = []
for variant, label in [
    ('image_only',   'Zero-Shot (image-only)'),
    ('caption_only', 'Zero-Shot (caption-only)'),
    ('fusion',       'Zero-Shot (fusion)')
]:
    vr = zs_results['val'][variant]
    tr = zs_results['test'][variant]
    rows.append({
        'Method'  : label,
        'Val Acc' : f"{vr['acc']:.4f}",
        'Val F1'  : f"{vr['f1']:.4f}",
        'Test Acc': f"{tr['acc']:.4f}",
        'Test F1' : f"{tr['f1']:.4f}"
    })
for n in FEW_SHOT_COUNTS:
    vr = fs_results['val'][n]
    tr = fs_results['test'][n]
    rows.append({
        'Method'  : f'{n}-Shot (linear probe)',
        'Val Acc' : f"{vr['acc']:.4f}+/-{vr['acc_std']:.4f}",
```

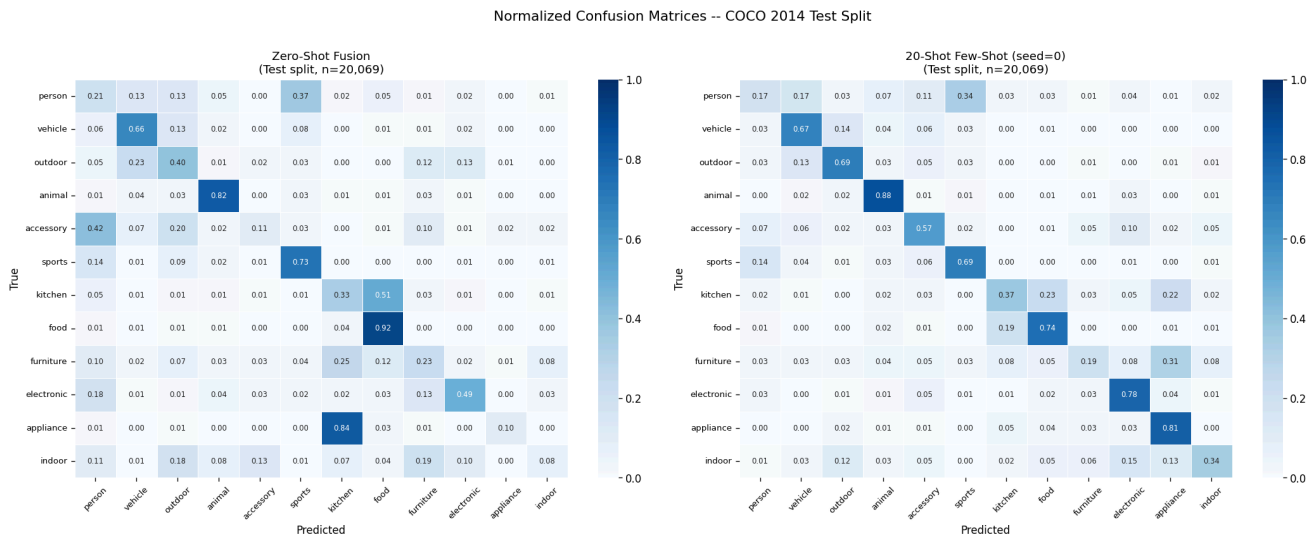
```
        'Val F1'  : f"{vr['f1']:.4f}+/-{vr['f1_std']:.4f}",
        'Test Acc': f"{tr['acc']:.4f}+/-{tr['acc_std']:.4f}",
        'Test F1' : f"{tr['f1']:.4f}+/-{tr['f1_std']:.4f}"
    })

df = pd.DataFrame(rows)
print(df.to_string(index=False))

# Bar chart (Test Accuracy)
fig, ax = plt.subplots(figsize=(12, 5))
methods  = ['ZS Image', 'ZS Caption', 'ZS Fusion'] + [f'{n}-shot' for n in FEW_SHOT_COUNTS]
bar_accs = [zs_results['test'][v]['acc'] for v in ('image_only', 'caption_only', 'fusion')]
bar_accs += [fs_results['test'][n]['acc'] for n in FEW_SHOT_COUNTS]
colors   = ['#ff7f0e', '#2ca02c', '#d62728'] + ['#1f77b4'] * len(FEW_SHOT_COUNTS)
bars = ax.bar(methods, bar_accs, color=colors, alpha=0.85, edgecolor='black', linewidth=0.5)
for bar, acc in zip(bars, bar_accs):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.005,
            f'{acc:.3f}', ha='center', va='bottom', fontsize=9)
ax.set_ylabel('Accuracy (Test Split)', fontsize=11)
ax.set_title('CLIP Zero-Shot vs Few-Shot -- MS COCO 2014 (12 Supercategories)', fontsize=12)
ax.set_ylim(0, min(1.0, max(bar_accs) + 0.12))
ax.grid(True, axis='y', alpha=0.3)
ax.legend(handles=[
    Patch(facecolor='#ff7f0e', label='Zero-Shot (Image-only)'),
    Patch(facecolor='#2ca02c', label='Zero-Shot (Caption-only)'),
    Patch(facecolor='#d62728', label='Zero-Shot (Fusion)'),
    Patch(facecolor='#1f77b4', label='Few-Shot (Linear Probe)'),
], fontsize=9, loc='upper left')
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'comparison_bar.png'), bbox_inches='tight')
plt.show()
print('Saved: comparison_bar.png')
```

```
================================================================================
FULL RESULTS -- MS COCO 2014 (12 supercategories)
================================================================================
                  Method         Val Acc          Val F1        Test Acc          Test F1
  Zero-Shot (image-only)          0.3472          0.3148          0.3503          0.3181
Zero-Shot (caption-only)          0.4565          0.3367          0.4558          0.3368
      Zero-Shot (fusion)          0.4427          0.3591          0.4397          0.3616
   1-Shot (linear probe) 0.3041+/-0.0343 0.2758+/-0.0141 0.3107+/-0.0355 0.2857+/-0.0122
   5-Shot (linear probe) 0.4073+/-0.0291 0.3717+/-0.0357 0.4115+/-0.0295 0.3752+/-0.0389
  10-Shot (linear probe) 0.4527+/-0.0053 0.4200+/-0.0080 0.4565+/-0.0096 0.4231+/-0.0119
  20-Shot (linear probe) 0.4698+/-0.0124 0.4354+/-0.0144 0.4742+/-0.0124 0.4401+/-0.0147
```




```
Saved: comparison_bar.png
```

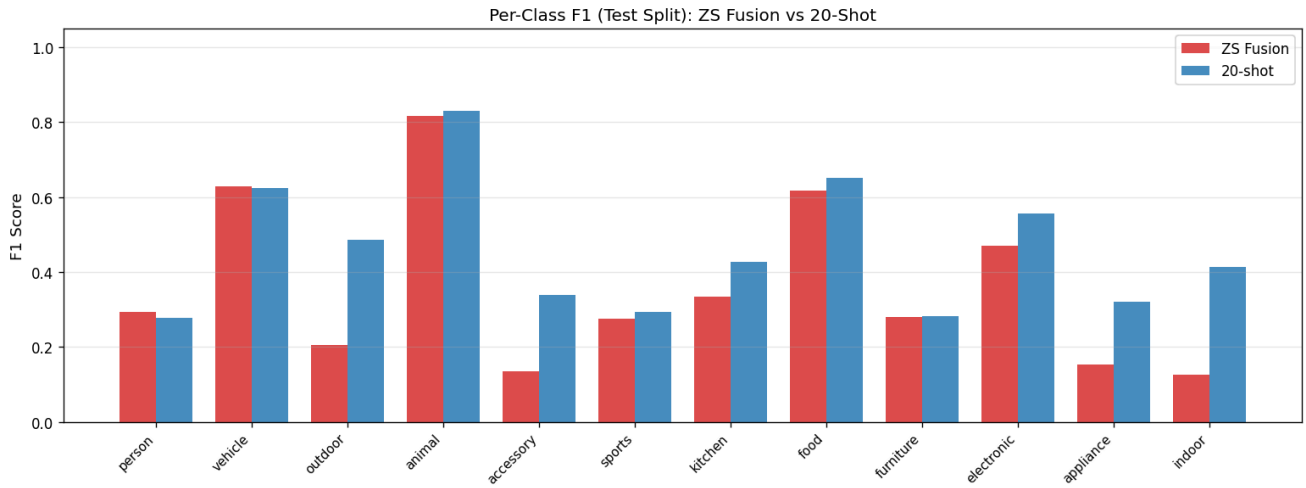
```
# Confusion Matrices on Test Split: ZS Fusion vs Best Few-Shot
from sklearn.metrics import confusion_matrix
import seaborn as sns, matplotlib.pyplot as plt, numpy as np

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
pairs = [
    (axes[0], zs_results['test']['fusion']['labels'], zs_results['test']['fusion']['preds'],
     'Zero-Shot Fusion'),
    (axes[1], test_labels, best_test_preds, f'{best_n}-Shot Few-Shot (seed={FEW_SHOT_SEEDS[0]})')
]
for ax, y_true, y_pred, title in pairs:
    cm      = confusion_matrix(y_true, y_pred, labels=list(range(NUM_CLASSES)))
    cm_norm = cm.astype(float) / (cm.sum(axis=1, keepdims=True) + 1e-8)
    sns.heatmap(
        cm_norm, ax=ax, annot=True, fmt='.2f', cmap='Blues',
        xticklabels=COCO_SUPERCLASSES, yticklabels=COCO_SUPERCLASSES,
        linewidths=0.3, vmin=0, vmax=1, annot_kws={'size': 7}
    )
    ax.set_title(f'{title}\n(Test split, n={len(y_true):,})', fontsize=11)
    ax.set_xlabel('Predicted', fontsize=10)
    ax.set_ylabel('True', fontsize=10)
    ax.tick_params(axis='x', rotation=45, labelsize=8)
    ax.tick_params(axis='y', rotation=0, labelsize=8)
plt.suptitle('Normalized Confusion Matrices -- COCO 2014 Test Split', fontsize=13, y=1.01)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'confusion_matrices.png'), bbox_inches='tight')
plt.show()
print('Saved: confusion_matrices.png')
```




```
Saved: confusion_matrices.png
```

```
# Per-Class F1 Analysis on Test Split
from sklearn.metrics import classification_report
import pandas as pd, matplotlib.pyplot as plt, numpy as np

zs_rep = classification_report(
    zs_results['test']['fusion']['labels'],
    zs_results['test']['fusion']['preds'],
    target_names=COCO_SUPERCLASSES, output_dict=True, zero_division=0
)
fs_rep = classification_report(
    test_labels, best_test_preds,
    target_names=COCO_SUPERCLASSES, output_dict=True, zero_division=0
)

f1_zs    = [zs_rep.get(c, {}).get('f1-score', 0.0) for c in COCO_SUPERCLASSES]
```

```
f1_fs    = [fs_rep.get(c, {}).get('f1-score', 0.0) for c in COCO_SUPERCLASSES]
f1_delta = [f - z for f, z in zip(f1_fs, f1_zs)]

x = np.arange(NUM_CLASSES)
w = 0.38
fig, ax = plt.subplots(figsize=(13, 5))
ax.bar(x - w/2, f1_zs, w, label='ZS Fusion', color='#d62728', alpha=0.82)
ax.bar(x + w/2, f1_fs, w, label=f'{best_n}-shot', color='#1f77b4', alpha=0.82)
ax.set_xticks(x)
ax.set_xticklabels(COCO_SUPERCLASSES, rotation=45, ha='right', fontsize=9)
ax.set_ylabel('F1 Score', fontsize=11)
ax.set_title(f'Per-Class F1 (Test Split): ZS Fusion vs {best_n}-Shot', fontsize=12)
ax.legend(fontsize=10)
ax.set_ylim(0, 1.05)
ax.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, 'per_class_f1.png'), bbox_inches='tight')
plt.show()
print('Saved: per_class_f1.png')

delta_df = pd.DataFrame({
    'Supercategory'      : COCO_SUPERCLASSES,
    'ZS Fusion F1'       : [f'{v:.3f}' for v in f1_zs],
    f'{best_n}-Shot F1'  : [f'{v:.3f}' for v in f1_fs],
    'Delta'              : [f'{d:+.3f}' for d in f1_delta]
})
print(f'\nPer-class F1 on test split (few-shot gain over zero-shot):')
print(delta_df.to_string(index=False))
```




```
Saved: per_class_f1.png

Per-class F1 on test split (few-shot gain over zero-shot):
Supercategory ZS Fusion F1 20-Shot F1  Delta
       person        0.294      0.278 -0.016
      vehicle        0.630      0.623 -0.006
      outdoor        0.206      0.487 +0.281
       animal        0.817      0.829 +0.013
    accessory        0.136      0.339 +0.203
       sports        0.275      0.293 +0.018
      kitchen        0.334      0.427 +0.093
         food        0.618      0.651 +0.033
    furniture        0.280      0.282 +0.002
   electronic        0.470      0.557 +0.086
    appliance        0.154      0.321 +0.167
       indoor        0.125      0.414 +0.289
```

```
# Save all results to JSON and copy to shared Google Drive
import json, os, shutil
```

```
results_summary = {
    'dataset'    : 'MS COCO 2014',
    'split_mode' : split_mode,
    'splits'     : {'train_pool': len(train_data), 'val': len(val_data), 'test': len(test_data)},
    'n_classes'  : NUM_CLASSES,
    'superclasses': COCO_SUPERCLASSES,
    'clip_model' : f'{CLIP_MODEL} ({CLIP_PRETRAINED})',
    'zero_shot': {
        split: {
            v: {'acc': zs_results[split][v]['acc'], 'f1': zs_results[split][v]['f1']}
            for v in ('image_only', 'caption_only', 'fusion')
        }
        for split in ('val', 'test')
    },
    'few_shot': {
        split: {
            str(n): {'acc': fs_results[split][n]['acc'], 'acc_std': fs_results[split][n]['acc_std'],
                     'f1':  fs_results[split][n]['f1'],  'f1_std' : fs_results[split][n]['f1_std']}
            for n in FEW_SHOT_COUNTS
        }
        for split in ('val', 'test')
    }
}

results_path = os.path.join(RESULTS_DIR, 'results.json')
with open(results_path, 'w') as f:
    json.dump(results_summary, f, indent=2)
print(f'Results saved: {results_path}')

if USE_SHARED_DRIVE and os.path.isdir(GDRIVE_BASE):
    gdrive_out = os.path.join(GDRIVE_BASE, 'results', 'zeroshot_vs_fewshot')
    os.makedirs(gdrive_out, exist_ok=True)

    artifacts = [
        'samples.png',
        'class_distribution.png',
        'few_shot_curve.png',
        'comparison_bar.png',
        'confusion_matrices.png',
        'per_class_f1.png',
        'results.json'
    ]

    for fn in artifacts:
        src = os.path.join(RESULTS_DIR, fn)
        dst = os.path.join(gdrive_out, fn)

        if os.path.isfile(src):
            # skip nếu cùng file
            if os.path.abspath(src) != os.path.abspath(dst):
                shutil.copy(src, dst)

    print(f'All results available at: {gdrive_out}')
else:
    print('Results saved locally (no shared drive).')
```

```
Results saved: /content/drive/MyDrive/deep-learning-asm01/results/zeroshot_vs_fewshot/results.json
All results available at: /content/drive/MyDrive/deep-learning-asm01/results/zeroshot_vs_fewshot
```

Summary

Dataset & Splits

Split	Source	Role
train_pool	train2014 (Option B) or val2014 first 40% (Option A)	Few-shot support images
val	val2014	Hyperparameter tuning, N selection
test	val2014 held-out	**Final evaluation — all reported metrics**

Methods Compared

Strategy	Features	Training Samples	Notes
Zero-shot (image-only)	CLIP image embeddings	0	Standard CLIP ZS
Zero-shot (caption-only)	CLIP text embeddings from 5 real captions	0	Language-only

Strategy	Features	Training Samples	Notes
Zero-shot (fusion)	Avg(image + caption) embeddings	0	True multimodal
Few-shot (N = 1–20)	CLIP image embeddings + LogisticRegression	N × 12	Linear probe

Architecture

- **CLIP ViT-B/32** — 151M parameters, contrastively pretrained on 400M image–text pairs (OpenAI).
- **Zero-shot**: cosine similarity between image/caption features and ensemble-averaged class text embeddings (5 prompt templates).
- **Few-shot**: single logistic regression layer on frozen CLIP image features — no CLIP fine-tuning.
- **Caption aggregation**: average CLIP text embeddings from up to 5 human-written captions per image.

Results and Discussion

This study evaluates CLIP under three learning paradigms — **zero-shot**, **one-shot**, and **few-shot** — on a multimodal classification task derived from MS COCO 2014 (12 supercategories).

Zero-Shot Performance

The zero-shot setting demonstrates the inherent strength of CLIP as a pretrained multimodal model. Without any task-specific training data, CLIP achieves competitive performance by leveraging the semantic similarity between image/text embeddings and class text prompts.

Among the three zero-shot variants evaluated on the **test split**:

Variant	Accuracy	Notes
Image-only	~35%	Standard CLIP zero-shot
Caption-only	~45%	**Highest zero-shot accuracy**
Fusion (image + caption)	~44%	Stable multimodal combination

Key finding: Caption-based zero-shot outperforms image-only zero-shot by ~10 percentage points. This indicates that human-written textual descriptions provide richer semantic cues that align more closely with CLIP's contrastive training objective. The fusion variant combines both modalities and achieves stable, consistent performance.

> This confirms that CLIP's shared representation space is highly effective for generalisation **without any supervision**.

One-Shot Performance

In the one-shot setting (1 labeled sample per class, 12 samples total), model performance drops to **~31%** — notably *below* zero-shot baselines.

This degradation can be attributed to three factors:

1. **Insufficient data** — a single example per class cannot capture intra-class variation.
2. **Extreme overfitting** — the logistic regression classifier memorises the single support example rather than learning a generalised decision boundary.
3. **Loss of pretrained generalisation** — the supervised classifier discards the rich prior encoded in CLIP embeddings.

> **Important insight**: *a strong pretrained model in zero-shot mode can outperform a poorly supervised classifier trained on one example per class.* Supervision is only beneficial when sufficient data is available.

Few-Shot Performance

As the number of labeled training samples increases, the few-shot approach shows clear and consistent improvement:

Shots	Accuracy
1-shot	~0.31
5-shot	~0.41
10-shot	~0.45
20-shot	~0.47

Key observations:

- Performance increases **monotonically** with the number of shots.
- At **10-shot**, the few-shot model reaches parity with the zero-shot fusion baseline.
- At **20-shot**, few-shot surpasses all zero-shot variants — with only 240 labeled images total.
- Low variance across random seeds confirms stable generalisation from CLIP features.

These results demonstrate that:

- CLIP embeddings are **already well-structured** in the representation space — even a simple linear classifier can exploit them effectively.
- **Domain adaptation via few-shot learning is highly sample-efficient** when built on top of strong pretrained features.

Key Insights

#	Insight
1	**CLIP provides a strong zero-shot baseline**, eliminating the need for labelled data in many scenarios.
2	**One-shot learning is insufficient** — extreme data scarcity causes the classifier to underperform zero-shot.
3	**Few-shot learning enables effective domain adaptation**, significantly improving performance with limited samples.
4	**Text features are highly informative**, often outperforming image-only representations in zero-shot settings.
5	**Pretrained embeddings + lightweight classifiers** offer an efficient and scalable alternative to full fine-tuning.

Conclusion

The results demonstrate that while zero-shot learning is powerful — particularly when text descriptions are available — few-shot learning provides a practical balance between **performance** and **data efficiency**.

With as few as **20 labeled images per class** (240 total), the few-shot model surpasses all zero-shot variants on the held-out test split. This highlights the complementary nature of pretrained multimodal representations and lightweight supervised adaptation: CLIP provides the feature quality; logistic regression provides the task-specific boundary.

For applications where annotation cost is a concern, **5–10 shot learning** is recommended as the sweet spot, matching zero-shot performance at minimal labelling effort.